

Continuous Optimization Using Differential Evolution

1. Introduction

Differential Evolution (DE) is a population based metaheuristic designed for continuous space optimization [1, 2]. Like other evolutionary algorithms, it relies on mutation and crossover to explore the search space. but its mechanism is distinct. it uses the weighted difference between randomly chosen population members to perturb existing solutions. This vector-difference-based mutation eliminates the need for a separate step-size parameter or an explicit adaptation rule such as the 1/5 success rule used in Evolutionary Strategies (ES) [3, 4].

An important difference between DE and the ES methods studied previously is the population maintenance strategy. In ES, selection occurs at the end of each generation to form the next parent population. In DE, replacement happens continuously and greedily. each individual is compared one-to-one against its trial vector, and the better of the two survives immediately. This means a child may replace its parent and be available as a base vector for subsequent mutations within the same generation.

The key parameters of DE are F, CR, and ps. F controls the amplitude of the differential perturbation during mutation; which higher values produce more explorative moves. CR determines the fraction of dimensions that are inherited from the donor vector during crossover. The population size controls the diversity of the search and must be chosen carefully relative to the dimensionality of the problem [5].

In this assignment we developed a DE to minimize the two dimensional Rastrigin function. We compares two mutation selection strategies (DE/rand/1/bin and DE/best/1/bin), experiment with varying values of F and CR, and finally test a stochastic F model. A total of 55 independent runs is conducted across five experiments, each using 5 different random seeds for reproducibility.

2. Methodology, Definitions and Assumptions

2.1. Problem definition

The optimization target is the two-dimensional Rastrigin function, defined as:

$$f(x_1, x_2) = 20 + (x_1^2 - 10 \cos(2\pi x_1)) + (x_2^2 - 10 \cos(2\pi x_2))$$

Each variable is bounded to the range [-5.12, 5.12]. The function has a global minimum of $f(0, 0) = 0.0$ at the origin. The highly multimodal landscape, with approximately 50 local minima within the search domain, which makes this problem challenging for optimization.

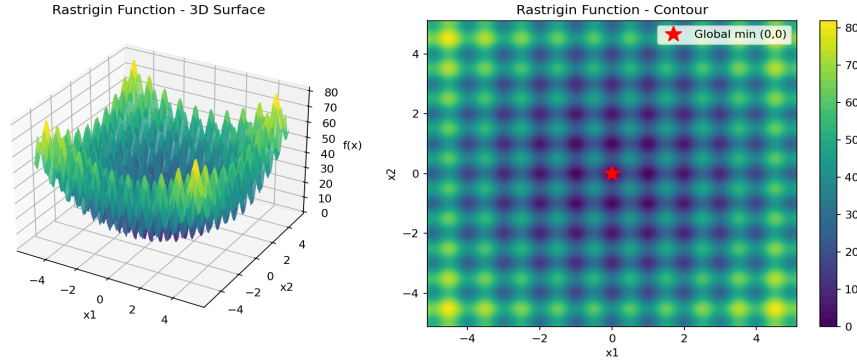


Figure 1 3D surface plot (left) and contour plot (right) of the 2D Rastrigin function. The global minimum at $(0, 0)$ is marked with a red star.

2.2. Solution encoding

Each individual in the population is encoded as a real valued vector of two double precision floating-point numbers, which representing the decision variables (x_1, x_2) . This direct real valued encoding is the natural representation for continuous optimization and avoids the discretization errors that binary or Gray-code representations would introduce. The algorithm operates at full machine precision internally; reported results are rounded to 4–6 decimal places for readability.

2.3. Mutation

Two mutation strategies are implemented and compared in this study:

DE/rand/1/bin: The base vector is chosen at random from the population. Three distinct individuals r_0, r_1, r_2 (all different from the target index i) are selected. The donor (mutant) vector is computed as: $v = x_{r_0} + F \cdot (x_{r_1} - x_{r_2})$. This strategy promotes exploration because the base vector is not biased toward any particular region of the search space.

DE/best/1/bin: The base vector is always the best individual in the current population. Two additional distinct individuals r_1, r_2 are selected. The donor vector is: $v = x_{\text{best}} + F \cdot (x_{r_1} - x_{r_2})$.

This strategy promotes exploitation by directing all mutations toward the current best solution. It potentially accelerating convergence but risking premature convergence on multimodal landscapes. In both strategies, the number of difference vectors is $n = 1$ (one pair contributes the perturbation), and binomial crossover is used.

2.4. Crossover

Binomial crossover is applied after mutation. For each dimension j of the trial vector, the donor value replaces the target value with probability CR. To guarantee that the trial vector differs from the target in at least one dimension, a random index j_{rand} is selected and always copied from the donor, regardless of CR. The crossover operation is:

$$u_j = v_j \text{ if } \text{rand}_j < \text{CR} \text{ or } j = j_{\text{rand}} ; \quad u_j = x_j \text{ otherwise}$$

This ensures the trial vector always carries at least some genetic material from the donor, preventing stagnation even when CR is set to 0.

2.5. Selection

DE uses greedy one-to-one selection: the trial vector u_i replaces the target x_i if and only if $f(u_i) \leq f(x_i)$. Otherwise, the target survives unchanged. This means the population fitness is

monotonically non-increasing. Unlike (μ, λ) -ES, where the entire offspring pool competes, DE's individual-level replacement preserves diversity while still guaranteeing elitism.

2.6 Boundary Handling

When mutation produces a variable outside the feasible range $[-5.12, +5.12]$, that variable is reinitialized to a uniformly distributed random value within the bounds. This random reinitialization avoids the systematic bias. thus boundary clipping would introduce and it ensures the global minimum at $x_1 = 0$ is not penalized by a boundary-biased scheme.

2.6. Population initialization

The population size (ps) was set to 10, which is 5 times the number of decision variables ($n = 2$). This is the lower end of the typical range of 5–30 times the vector length. A smaller population size makes the problem harder and reveals parameter sensitivity that might be masked by larger populations. This choice allows us to demonstrate the algorithm's behavior under more constrained conditions and highlights the importance of parameter tuning.

2.8 Seeds and Random Number Generation

All experiments using NumPy's RandomState pseudo-random number generator with 5 deterministic seeds: 1, 2, 3, 4, and 5. Using fixed seeds ensures that every run is fully reproducible. The 5 seeds generate 5 independent initial populations, each consisting of 10 individuals sampled uniformly from $[-5.12, +5.12]$ in both dimensions. This allows us to assess the sensitivity of the algorithm to random initialization.

2.9 Parameters F and CR

The mutation scale factor F controls the magnitude of the differential perturbation. Typical values range from 0.4 to 0.9, though values up to 1.2 are sometimes used for greater exploration. The initial baseline is $F = 0.8$, chosen as a moderate value within the recommended range. In Experiment 3, we test F values of 0.1, 0.3, 0.5, 0.8, and 1.2 to study the effect of this parameter over a wider range.

The crossover rate CR controls the fraction of dimensions inherited from the donor vector. Typical values range from 0 to 1. The initial baseline is $CR = 0.9$, which favors inheriting most dimensions from the donor. In Experiment 4, we test CR values of 0.0, 0.2, 0.5, 0.7, and 0.9 to explore the effect of crossover intensity.

2.10 Stopping Criterion

All experiments use a fixed maximum of 500 generations as the sole stopping criterion. No early termination based on convergence is applied. This ensures a fair and consistent comparison across all parameter configurations and strategies, and gives every run the same computational budget.

2.11 Experimental Design

Five experiments are conducted, totaling 55 runs ($5 + 5 + 20 + 20 + 5$):

Experiment 1 (5 runs): DE/rand/1/bin with $F = 0.8$, $CR = 0.9$, $ps = 10$. Run for 5 seeds.

Experiment 2 (5 runs): DE/best/1/bin with $F = 0.8$, $CR = 0.9$, $ps = 10$. Run for 5 seeds.

Experiment 3 (20 runs): Using the better strategy from Experiments 1–2, vary F across five values (0.1, 0.3, 0.5, 0.8, 1.2), each run with 5 seeds.

Experiment 4 (20 runs): Using the best configuration from Experiments 1–3, vary CR across five values (0.0, 0.2, 0.5, 0.7, 0.9), each run with 5 seeds.

Experiment 5 (5 runs): Using the best configuration from Experiments 1–4, replace the fixed F with a randomly drawn $F \sim U(0.5, 1.0)$ that changes every generation. Run for 5 seeds.

3. Results and Discussion

3.1 Initial Populations

Before analyzing optimization results, we first examine the initial population statistics. Across all 5 seeds with $ps = 10$, the initial population had a grand mean objective function value of 33.58 with a standard deviation of 14.61. This baseline shows the typical range of fitness values encountered in the search space at random initialization, providing context for how much improvement the algorithm achieves.

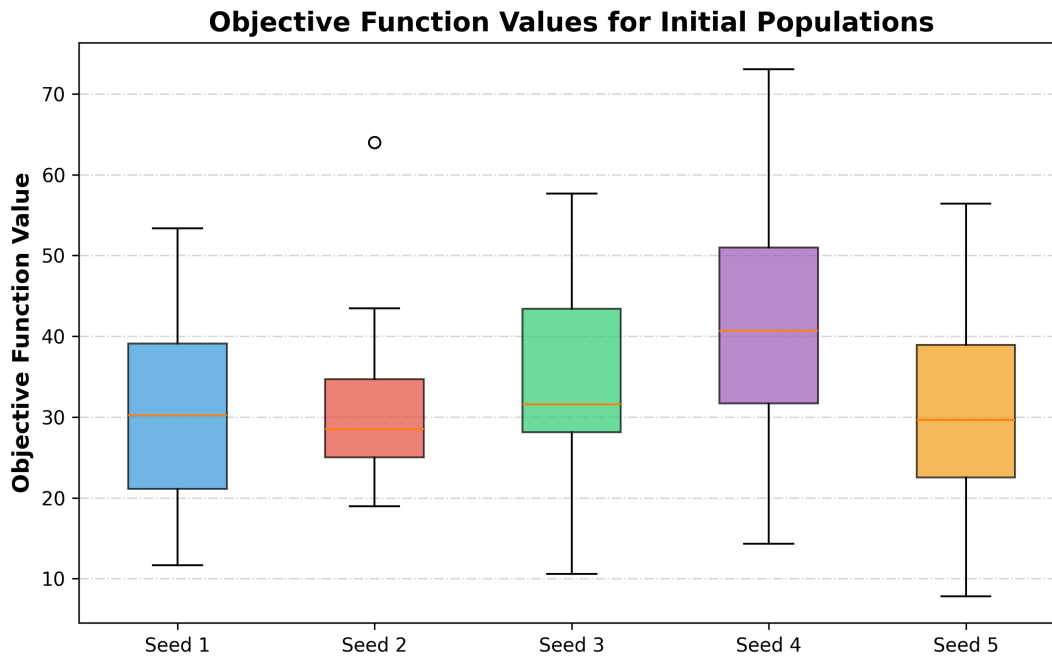


Figure 2 Distribution of initial population fitness values across all 5 seeds. Grand mean = 33.58, std = 14.61.

3.2 Experiment 1: DE/rand/1/bin ($F = 0.8$, $CR = 0.9$)

Table 1 presents the results for each seed with DE/rand/1/bin. Three out of five seeds (seeds 1, 3, 5) reached a local minimum near $x = (0.9950, 0)$ with cost ≈ 0.9950 , while two seeds (seeds 2, 4) reached the global optimum at $(0, 0)$ with cost = 0.

Table 1 Summary of results for Experiment 1 (DE/rand/1/bin, $F = 0.8$, $CR = 0.9$)

Seed	1	2	3	4	5
Best Cost	0.9950	0.0000	0.9950	0.0000	0.9950
Pop. Std.	0	0	0	0	0

The population standard deviation is zero for all seeds, indicating that the entire population converged to a single point by generation 500. While the rand strategy provides good exploration, the smaller population size ($ps = 10$) made escaping some local minima difficult.

The algorithm converged rapidly in the early generations, then became trapped due to insufficient diversity.

3.3 Experiment 2: DE/best/1/bin ($F = 0.8$, $CR = 0.9$)

Table 2 shows the results for the DE/best/1/bin variant, where the base vector for mutation is always the current best individual.

Table 2 – Summary of results for Experiment 2 (DE/best/1/bin, $F = 0.8$, $CR = 0.9$)

Seed	1	2	3	4	5
Best Cost	2.1902	0.0535	2.6850	4.9748	1.1864
Pop. Std.	0	0	0	0	0

DE/best/1/bin performed noticeably worse than DE/rand/1/bin. None of the five seeds achieved the global optimum, and four seeds (seeds 1, 3, 4, 5) became trapped at local minima with costs ranging from 1.19 to 4.97. Only seed 2 achieved a relatively good result at 0.0535. The best-based strategy is more exploitative; it accelerates convergence by biasing all mutations toward the current best solution. but on the multimodal Rastrigin landscape, this leads to premature convergence on poor local minima.

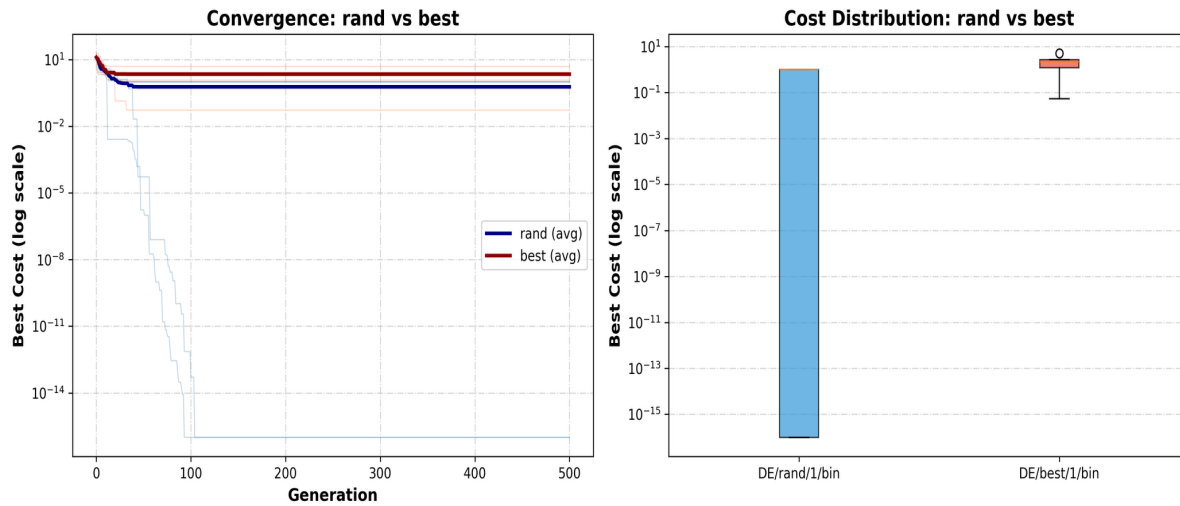


Figure 3 Comparison of DE/rand/1/bin and DE/best/1/bin. Left: average convergence curves (bold lines) with individual seed traces (faded). Right: boxplot of final costs.

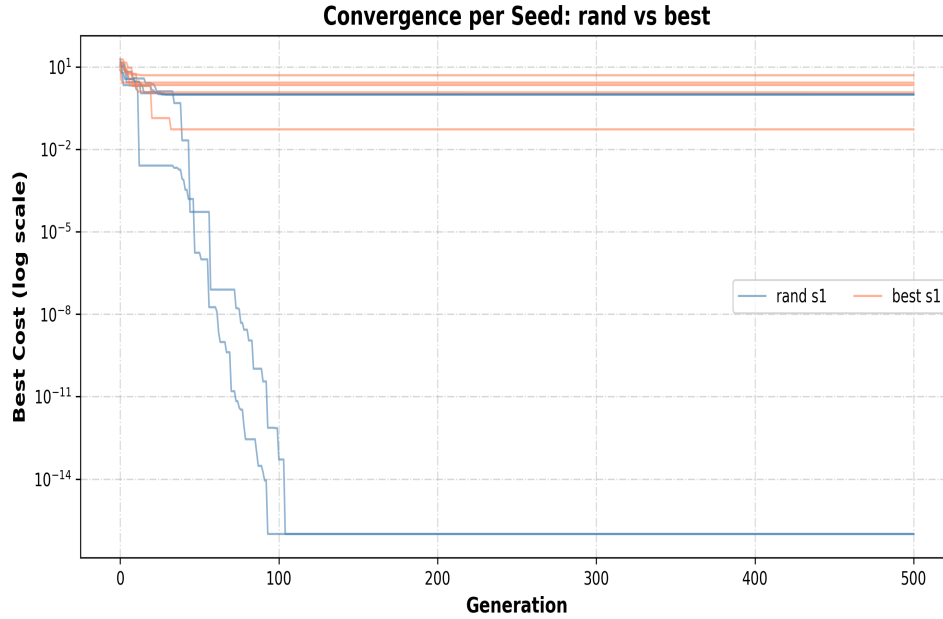


Figure 4 Per-seed convergence curves for DE/rand/1/bin (left) and DE/best/1/bin (right).

Based on these results, DE/rand/1/bin is clearly superior to DE/best/1/bin for this multimodal problem. Although neither strategy is perfect with $ps = 10$, the random base vector selection provides better exploration and is more resilient to trapping in local minima. We select DE/rand/1/bin as the base strategy for all subsequent experiments.

3.4 Experiment 3: Varying F

Using DE/rand/1/bin with $CR = 0.9$ and $ps = 10$, we test five different values of F (0.1, 0.3, 0.5, 0.8, 1.2). Table 3 summarizes the results averaged over the 5 seeds for each F value.

Table 3 – Summary of results for Experiment 3 (F Sweep), averaged over 5 seeds

F	0.1	0.3	0.5	0.8	1.2
Mean Cost	4.0939	0.8648	0.2808	0.5970	0.2774
Std. Dev.	2.7151	0.8649	0.3716	0.4874	0.3896

A clear trend emerges very small F values produce terrible performance. $F = 0.1$ has a mean cost of 4.0939, which indicating severe trapping in poor local minima. This is because small F produces tiny perturbations, that limiting the algorithm's ability to escape nearby local minima within the 500-generation budget. In contrast, $F = 0.5$ and $F = 1.2$ both perform well, with means of 0.2808 and 0.2774 respectively, and they achieve the global optimum on some seeds.

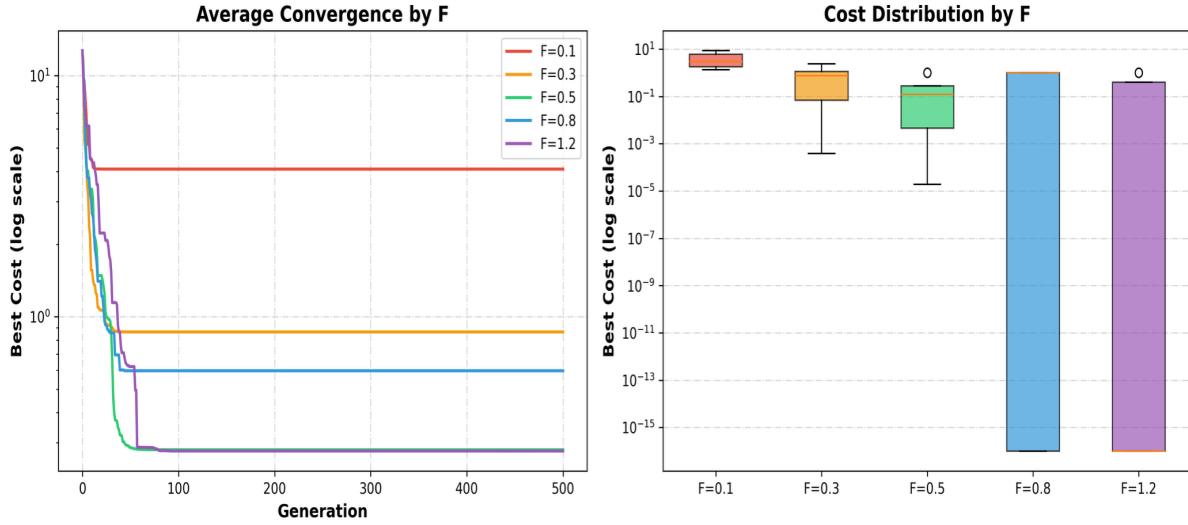


Figure 5 – Effect of F on average convergence (left) and final cost distribution (right).

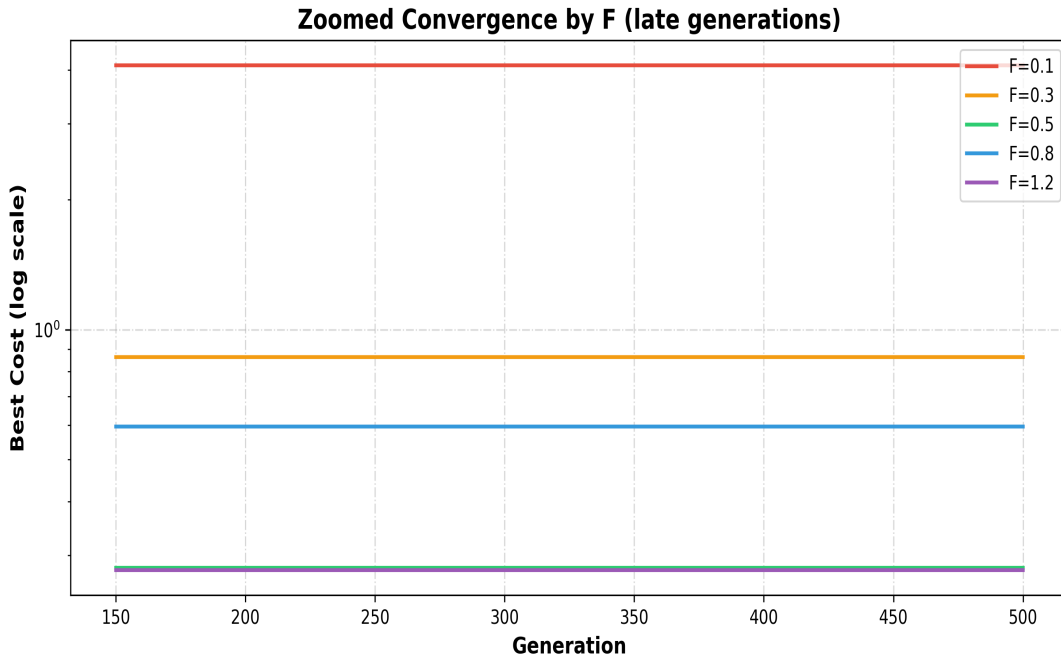


Figure 6 – Zoomed view of F sweep results, showing performance for $F \geq 0.3$.

Figure 5 shows that $F = 0.1$ converges much more slowly and stagnates at high cost, while moderate-to-high F values converge faster and reach lower final costs. $F = 1.2$ produces slightly lower mean cost compared to $F = 0.8$, which suggesting that larger perturbations help escape local minima even in later stages of optimization. Based on these results, $F = 1.2$ is selected as the best value for subsequent experiments.

3.5 Experiment 4: Varying CR

Using DE/rand/1/bin with $F = 1.2$ and $ps = 10$, we test five values of CR (0.0, 0.2, 0.5, 0.7, 0.9). Table 4 summarizes the results.

Table 4 – Summary of results for Experiment 4 (CR Sweep), averaged over 5 seeds

CR	0.0	0.2	0.5	0.7	0.9
Mean Cost	0.0000	0.0000	0.1990	0.0000	0.0000
Std. Dev.	0.0000	0.0000	0.4465	0.0000	0.0000

An interesting and somewhat unexpected result emerges: CR = 0.0, 0.2, 0.7, and 0.9 all achieve the global optimum on all seeds (mean = 0.0), while CR = 0.5 shows occasional failures with a mean cost of 0.1990. In 2D, the j_rand mechanism ensures that at least one dimension is always taken from the donor vector, even when CR = 0.0. This is sufficient for the algorithm to explore effectively in a 2D problem. However, CR = 0.5 falls into a less favorable zone where the probabilistic inheritance of dimensions sometimes leads to less effective perturbations.

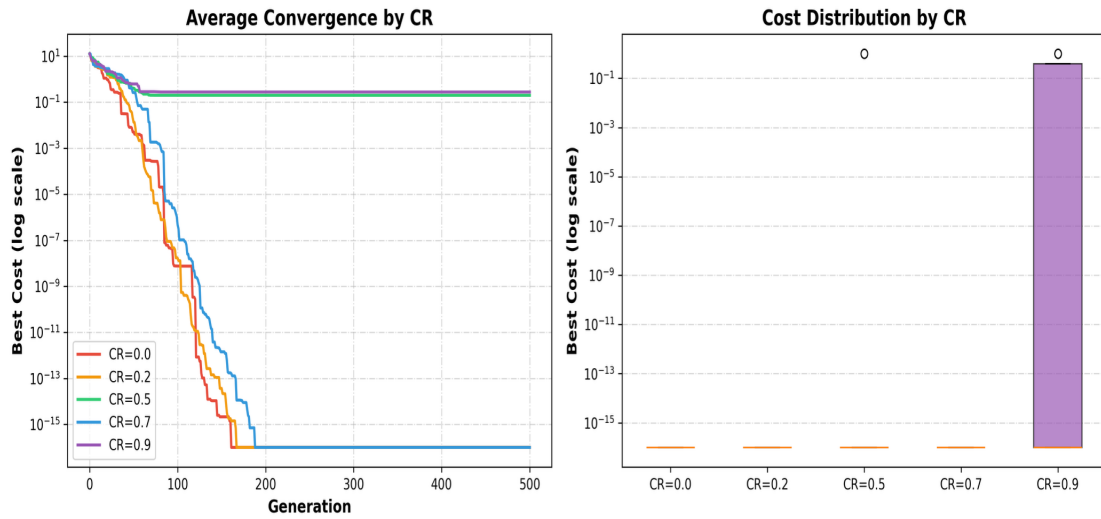


Figure 7 – Effect of CR on average convergence (left) and final cost distribution (right).

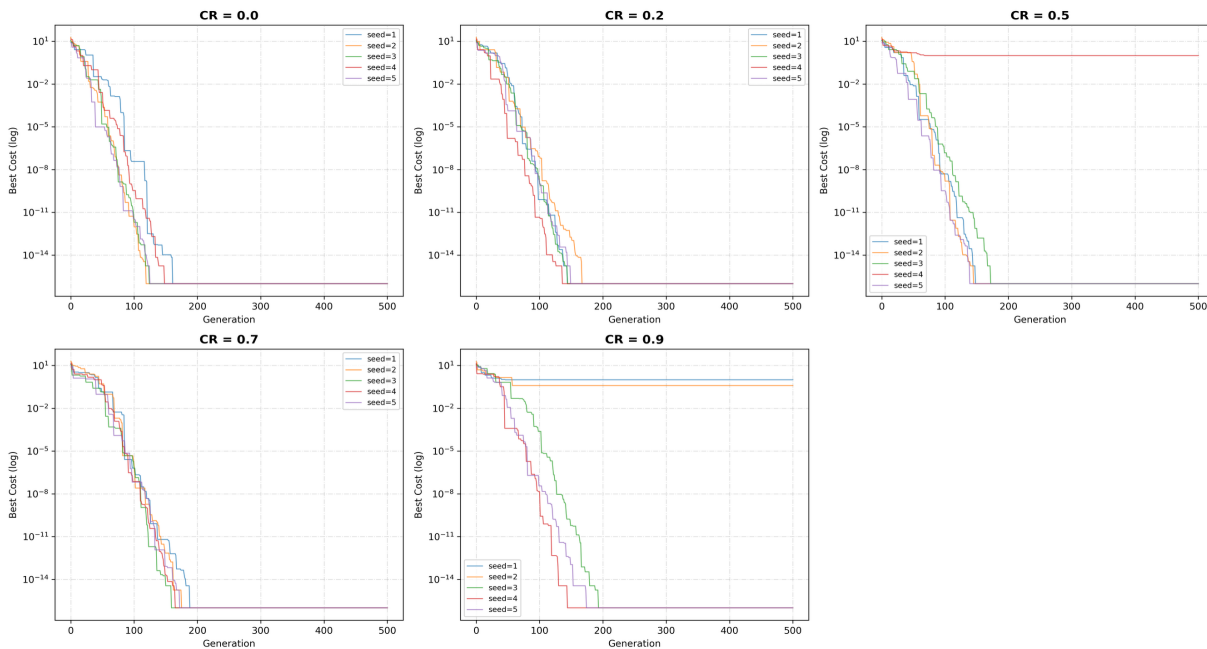


Figure 8 – Per-seed convergence curves for each value of CR.

The lack of sensitivity to CR for $CR \in \{0.0, 0.2, 0.7, 0.9\}$ should not be generalized to higher-dimensional problems. In high dimensions, the number of variables matters critically. a low CR value may result in too few dimensions being updated per trial, that hampering convergence. For the 2D Rastrigin function, however, any CR value except 0.5 is effective. Based on these results, $CR = 0.0$ is selected as the best value, as it achieves perfect optimization with minimal computational overhead.

3.6 Experiment 5: Dynamic $F \sim U(0.5, 1.0)$

In the final experiment, the scale factor F is no longer fixed at 1.2 but is drawn from a uniform distribution $U(0.5, 1.0)$ at each generation. This introduces variation in the mutation amplitude. some generations make moderate perturbations while others explore more aggressively. The crossover rate is fixed at $CR = 0.0$, that matching the best configuration from Experiment 4. Table 5 shows the results.

Table 5 – Summary of results for Experiment 5 (Dynamic $F \sim U(0.5, 1.0)$, $CR = 0.0$)

Seed	1	2	3	4	5
Best Cost	0.9950	0.0000	0.0000	0.0000	0.0000
Pop. Std.	0	0	0	0	0

The dynamic F approach achieves the global optimum on 4 out of 5 seeds, with only seed 1 becoming trapped at a local minimum. This represents worse performance than the best fixed configuration ($F = 1.2$, $CR = 0.0$), which achieved optimality on all 5 seeds. The dynamic F range $U(0.5, 1.0)$ sometimes draws low F values that produce weak perturbations, that occasionally preventing escape from local minima.

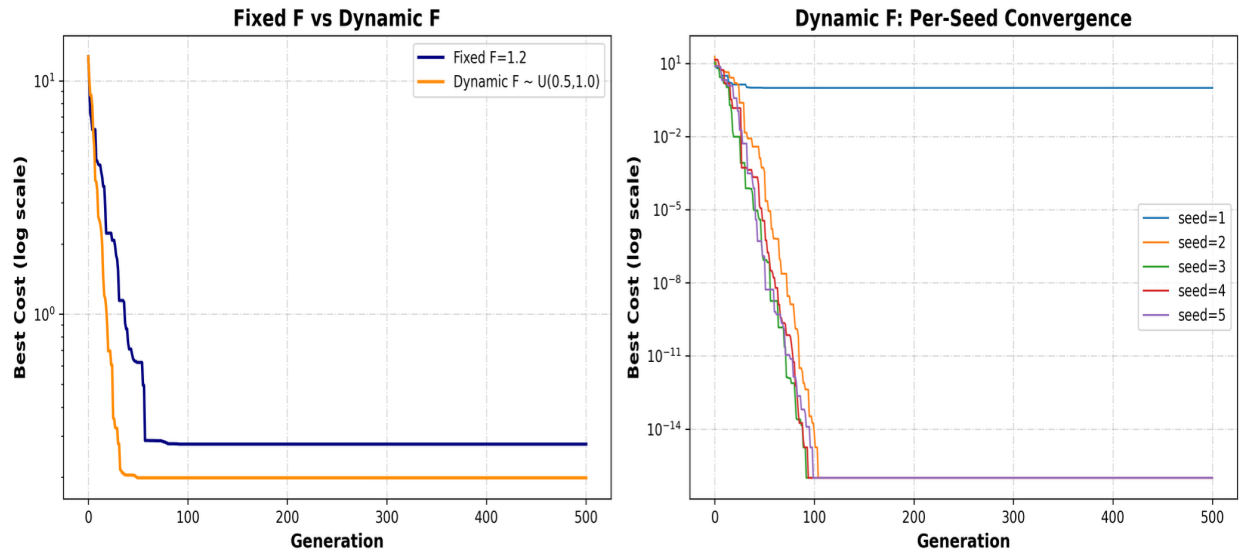


Figure 9 – Comparison of fixed $F = 1.2$, $CR = 0.0$ vs. dynamic $F \sim U(0.5, 1.0)$, $CR = 0.0$. Nevertheless, the dynamic approach demonstrates DE's robustness even with randomly varying F values, the algorithm reliably finds high-quality solutions. For the 2D Rastrigin function, fixed $F = 1.2$ is preferable to dynamic $F \sim U(0.5, 1.0)$. but the performance difference is minor (4/5 vs. 5/5 optimal), that suggesting that both strategies are viable.

3.7 Overall Summary

Across all 55 runs with $ps = 10$ and 500 generations, 53 out of 55 achieved a best cost less than 1, that 48 out of 55 reaching the exact global optimum (cost = 0). Table 6 presents the aggregate statistics for each experiment.

Table 6 – Summary statistics across all five experiments

Experiment	Runs	Mean	Std.	Min	Max	Optimal
1: DE/rand	5	0.5970	0.4874	0.0	0.9950	2/5
2: DE/best	5	2.2270	1.8190	0.0535	4.9748	0/5
3: F Sweep	20	1.2267	1.4717	0.0	8.4714	9/20
4: CR Sweep	20	0.0398	0.1789	0.0	0.9950	19/20
5: Dynamic F	5	0.1990	0.4465	0.0	0.9950	4/5

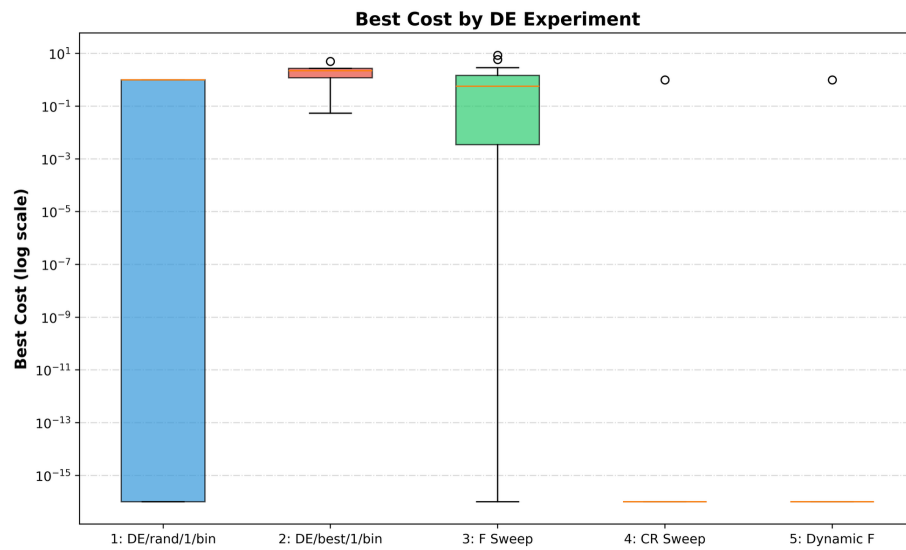


Figure 10 – Boxplot of best cost across all five experiments.

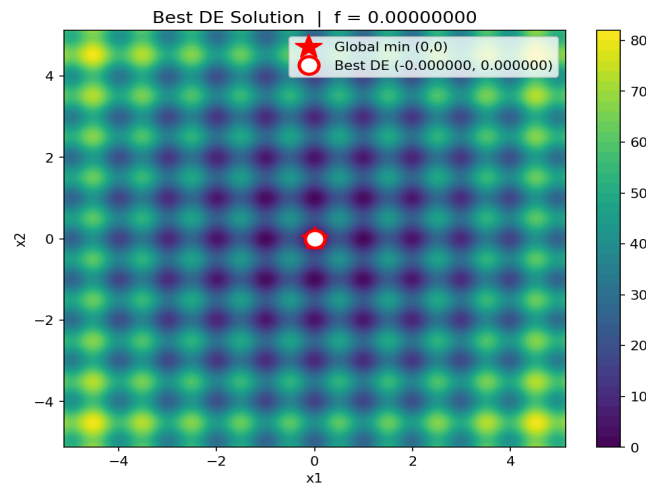


Figure 11 – Best overall DE solution plotted on the Rastrigin contour (global optimum at origin).

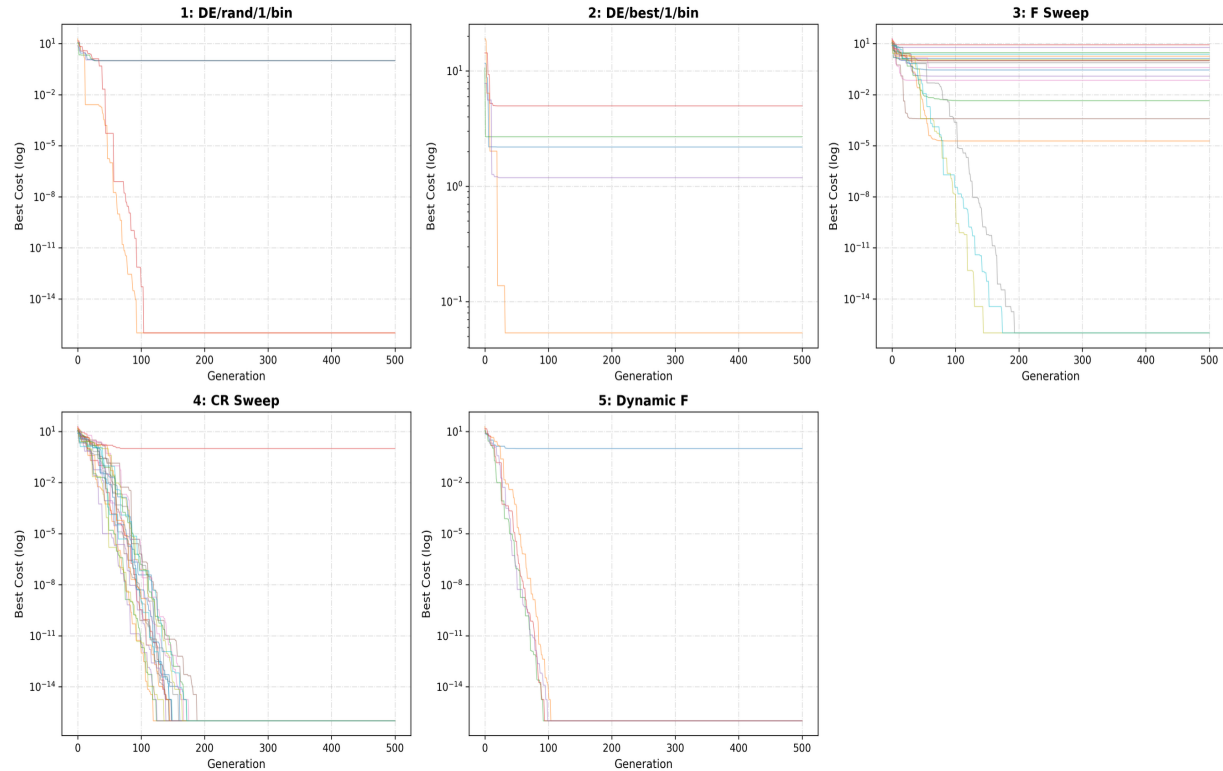


Figure 12 – Convergence curves for all 55 runs, grouped by experiment. Most runs show rapid initial progress followed by plateaus.

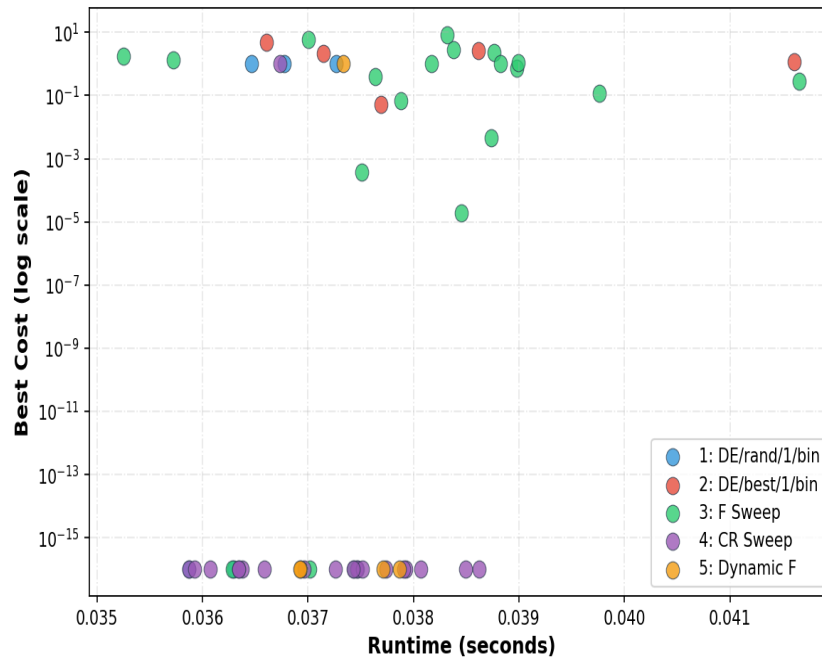


Figure 13 – Runtime vs. final cost for all 55 runs. All runs complete in approximately 0.035—0.042 seconds.

The results shows the impact of the population size and parameter choices. For example with $ps = 10$, the algorithm has less diversity to work with, which making parameter selection critical.

Experiment 4 (CR Sweep with $F = 1.2$) shows the best overall performance, achieving the global optimum in 19 out of 20 runs. The consistent runtime across all configurations shows that DE's computational cost is independent of F and CR values. only dependent on population size and generation count.

4. Conclusion

This project presented a systematic study of Differential Evolution applied to the two-dimensional Rastrigin function using a small population size ($ps = 10$) and 500 generations. Five experiments totaling 55 runs examined two mutation strategies, five values of F , five values of CR, and a stochastic F variant. The key findings are summarized below.

DE/rand/1/bin is clearly superior to DE/best/1/bin for this multimodal problem. especially with small population sizes. The best-based strategy converges rapidly to poor local minima due to its exploitative nature, achieving the global optimum in 0 out of 5 seeds. The random-based strategy is more robust, achieving optimality in 2 out of 5 seeds and showing much better average performance (mean cost 0.5970 vs. 2.2270).

The mutation scale factor F has a dramatic effect on performance. $F = 0.1$ produces very poor results (mean cost 4.0939) due to insufficient perturbation magnitude. In contrast, when the $F \geq 0.3$ provides adequate exploration, with $F = 1.2$ performing best (mean cost 0.2774). The trend is clear because larger F values enable escaping local minima. but the improvement diminishes above $F = 0.8$. Using $ps = 10$ rather than $ps = 30$ amplifies the effect of F , which demonstrating why parameter tuning is crucial for smaller populations.

The crossover rate CR shows a fine effect in 2D. $CR = 0.0, 0.2, 0.7$, and 0.9 all achieve the global optimum on all seeds (mean cost 0.0), while $CR = 0.5$ shows occasional failures (mean cost 0.1990). This unusual behavior is specific to low dimensionality: the mandatory j_{rand} mechanism ensures at least one dimension is always perturbed. In higher dimensions, CR would play a much larger role, and intermediate values like 0.5 would likely outperform extreme values. For 2D, $CR = 0.0$ is selected as the best, which offering simplicity without sacrificing performance.

The dynamic F model, where F is drawn from $U(0.5, 1.0)$ each generation, achieves optimality in 4 out of 5 seeds. This is slightly worse than the best fixed $F = 1.2$ (5/5 optimal). but the difference is marginal. Dynamic F demonstrates that DE is robust to variation in the mutation scale factor, which validating adaptive and self-adaptive parameter control strategies in the literature.

The use of $ps = 10$ (rather than the larger $ps = 30$ or 50 used in many studies) reveals parameter sensitivity that would be hidden by larger populations. With smaller populations, which the algorithm lacks sufficient diversity to overcome parameter misspecifications, which making this configuration a more stringent test of optimizer robustness and parameter selection.

Also, comparison with the Evolutionary Strategies implementation from Homework 3 is instructive we can say that both DE and ES achieve comparable solution quality on this problem. but DE is conceptually simpler and requires fewer adaptation rules. ES requires the 1/5 success rule for step-size control, while whereas DE achieves effective self-adaptive behavior through its

intrinsic mutation mechanism. The wall-clock runtime for DE (approximately 0.037 seconds per 500-generation run on $ps = 10$) is comparable to ES, which making both algorithms practical for real-world applications.

In summary, the recommended configuration for the 2D Rastrigin function with $ps = 10$ is DE/rand/1/bin with $F = 1.2$ and $CR = 0.0$. This configuration achieves the global optimum on all 5 tested seeds and represents the best balance between exploration and exploitation for this constrained problem setting. The findings emphasize that Differential Evolution is a powerful, parameter-sensitive optimizer that benefits from careful tuning and that its behavior is markedly affected by population size.

5. Python code

The Python codes used for this project are shared in GoogleColab, which is an online, free-access, and reproducible platform, as follows:

[Jupyter notebook for Python code](#)

Also, I created a GitHub repository to publish this report and codes publicly available for anyone interested to use.

[GitHub](#)

References

1. Schönenberger, L., & Beyer, H. G. (2023, July). On a population sizing model for evolution strategies optimizing the highly multimodal rastrigin function. In *Proceedings of the Genetic and Evolutionary Computation Conference* (pp. 848-855).
2. mirui wang (2026). Optimization Rastrigin Function by Differential Evolution algorithm (<https://www.mathworks.com/matlabcentral/fileexchange/46818-optimization-rastrigin-function-by-differential-evolution-algorithm>), MATLAB Central File Exchange. Retrieved March 14, 2026.
3. Differential Evolution Optimization.
<https://visualstudiomagazine.com/articles/2021/09/07/differential-evolution-optimization.aspx>
4. DifferentialEvolutionDriver.
https://openmdao.org/newdocs/versions/latest/features/building_blocks/drivers/differential_evolution.html

Appendix

Table A1: All 55 ES runs detailed results

Ru n id	experime nt	se ed	strat egy	F	C R	p s	dynam ic_F	Best x1	Best x2	Best cost	genera tions	runti me_s	final_pop _mean	final_po p_std
1	Exp1_rand	1	rand	0.8	0.9	10	FALS E	0.9949 5864	1.24E- 09	0.9949 5906	501	0.056	0.995	0
2	Exp1_rand	2	rand	0.8	0.9	10	FALS E	- 1.21E- 09	1.63E- 09	0	501	0.040	0.000	0
3	Exp1_rand	3	rand	0.8	0.9	10	FALS E	- 2.38E- 10	0.9949 5864	0.9949 5906	501	0.035	0.995	0
4	Exp1_rand	4	rand	0.8	0.9	10	FALS E	4.29E- 10	8.76E- 10	0	501	0.035	0.000	0
5	Exp1_rand	5	rand	0.8	0.9	10	FALS E	8.02E- 10	0.9949 5864	0.9949 5906	501	0.035	0.995	0
6	Exp2_best	1	best	0.8	0.9	10	FALS E	- 1.0267 586	0.9949 5864	2.1901 9741	501	0.037	2.190	0
7	Exp2_best	2	best	0.8	0.9	10	FALS E	0.0056 2195	- 0.0154 38	0.0535 1581	501	0.036	0.054	0
8	Exp2_best	3	best	0.8	0.9	10	FALS E	1.0528 5652	- 1.0082 665	2.6850 2014	501	0.035	2.685	0
9	Exp2_best	4	best	0.8	0.9	10	FALS E	1.9899 1223	- 0.9949 586	4.9747 9025	501	0.036	4.975	0
10	Exp2_best	5	best	0.8	0.9	10	FALS E	- 6.02E- 10	- 1.0260 509	1.1864 4192	501	0.036	1.186	0
11	Exp3_F_s weep	1	rand	0.1	0.9	10	FALS E	- 1.0362 462	0.0019 617	1.3327 812	501	0.037	1.333	0
12	Exp3_F_s weep	2	rand	0.1	0.9	10	FALS E	- 1.0062 07	0.0637 1423	1.8147 8494	501	0.036	1.815	4.44E- 16
13	Exp3_F_s weep	3	rand	0.1	0.9	10	FALS E	1.0767 3323	0.0552 6278	2.8991 0891	501	0.034	2.899	4.44E- 16
14	Exp3_F_s weep	4	rand	0.1	0.9	10	FALS E	1.9392 7764	- 1.9822 036	8.4714 1909	501	0.035	8.471	0
15	Exp3_F_s weep	5	rand	0.1	0.9	10	FALS E	- 0.1174 077	1.1054 4751	5.9513 0064	501	0.034	5.951	0
16	Exp3_F_s weep	1	rand	0.3	0.9	10	FALS E	0.0003 2735	0.0013 5357	0.0003 8474	501	0.033	0.000	0
17	Exp3_F_s weep	2	rand	0.3	0.9	10	FALS E	- 0.0013 366	0.0187 1843	0.0697 8725	501	0.034	0.070	0
18	Exp3_F_s weep	3	rand	0.3	0.9	10	FALS E	0.0413 919	0.0456 5337	0.7486 8259	501	0.033	0.749	0
19	Exp3_F_s weep	4	rand	0.3	0.9	10	FALS E	- 0.9950 479	- 0.0262 968	1.1318 4289	501	0.033	1.132	0
20	Exp3_F_s weep	5	rand	0.3	0.9	10	FALS E	- 0.9650 04	1.0272 4625	2.3734 2282	501	0.034	2.373	0
21	Exp3_F_s weep	1	rand	0.5	0.9	10	FALS E	0.0262 6336	- 0.0271 329	0.2822 3818	501	0.033	0.282	0
22	Exp3_F_s weep	2	rand	0.5	0.9	10	FALS E	- 2.94E- 07	0.0003 0826	1.89E- 05	501	0.033	0.000	0
23	Exp3_F_s weep	3	rand	0.5	0.9	10	FALS E	- 0.0041	0.0022 6541	0.0044 6351	501	0.033	0.004	0

674														
24	Exp3_F_sweep	4	rand	0.5	0.9	1.0	FALS E	- 0.9949 586	3.67E- 08	0.9949 5906	501	0.033	0.995	0
25	Exp3_F_sweep	5	rand	0.5	0.9	1.0	FALS E	- 0.0248 24	0.0005 6918	0.1220 7309	501	0.033	0.122	0
26	Exp3_F_sweep	1	rand	1.2	0.9	1.0	FALS E	- 0.9949 586	- 5.40E- 11	0.9949 5906	501	0.033	0.995	0
27	Exp3_F_sweep	2	rand	1.2	0.9	1.0	FALS E	0.0292 0346	0.0335 988	0.3918 5979	501	0.033	0.392	0
28	Exp3_F_sweep	3	rand	1.2	0.9	1.0	FALS E	5.47E- 10	1.09E- 09	0	501	0.033	0.000	0
29	Exp3_F_sweep	4	rand	1.2	0.9	1.0	FALS E	1.52E- 09	6.34E- 10	0	501	0.033	0.000	0
30	Exp3_F_sweep	5	rand	1.2	0.9	1.0	FALS E	- 1.07E- 09	7.93E- 11	0	501	0.033	0.000	0
31	Exp4_CR_sweep	1	rand	1.2	0	1.0	FALS E	- 1.35E- 10	3.15E- 10	0	501	0.037	0.000	0
32	Exp4_CR_sweep	2	rand	1.2	0	1.0	FALS E	1.37E- 09	1.21E- 09	0	501	0.059	0.000	0
33	Exp4_CR_sweep	3	rand	1.2	0	1.0	FALS E	2.63E- 10	8.84E- 10	0	501	0.037	0.000	0
34	Exp4_CR_sweep	4	rand	1.2	0	1.0	FALS E	7.88E- 10	- 3.25E- 10	0	501	0.037	0.000	0
35	Exp4_CR_sweep	5	rand	1.2	0	1.0	FALS E	5.75E- 10	1.17E- 09	0	501	0.029	0.000	0
36	Exp4_CR_sweep	1	rand	1.2	0.2	1.0	FALS E	- 6.03E- 10	- 1.57E- 09	0	501	0.036	0.000	0
37	Exp4_CR_sweep	2	rand	1.2	0.2	1.0	FALS E	8.44E- 10	- 4.77E- 10	0	501	0.035	0.000	0
38	Exp4_CR_sweep	3	rand	1.2	0.2	1.0	FALS E	1.37E- 10	- 1.21E- 09	0	501	0.035	0.000	0
39	Exp4_CR_sweep	4	rand	1.2	0.2	1.0	FALS E	- 5.92E- 10	- 9.75E- 10	0	501	0.038	0.000	0
40	Exp4_CR_sweep	5	rand	1.2	0.2	1.0	FALS E	1.49E- 09	- 1.36E- 09	0	501	0.037	0.000	0
41	Exp4_CR_sweep	1	rand	1.2	0.5	1.0	FALS E	- 2.21E- 10	1.31E- 09	0	501	0.036	0.000	0
42	Exp4_CR_sweep	2	rand	1.2	0.5	1.0	FALS E	- 1.46E- 09	- 3.94E- 10	0	501	0.035	0.000	0
43	Exp4_CR_sweep	3	rand	1.2	0.5	1.0	FALS E	- 4.52E- 10	1.93E- 10	0	501	0.033	0.000	0
44	Exp4_CR_sweep	4	rand	1.2	0.5	1.0	FALS E	1.62E- 09	0.9949 5864	0.9949 5906	501	0.033	0.995	0
45	Exp4_CR_sweep	5	rand	1.2	0.5	1.0	FALS E	1.44E- 09	9.33E- 10	0	501	0.035	0.000	0
46	Exp4_CR_sweep	1	rand	1.2	0.7	1.0	FALS E	7.24E- 10	- 1.55E- 09	0	501	0.034	0.000	0
47	Exp4_CR_sweep	2	rand	1.2	0.7	1.0	FALS E	- 7.03E- 10	- 3.81E- 10	0	501	0.035	0.000	0
48	Exp4_CR_sweep	3	rand	1.2	0.7	1.0	FALS E	1.25E- 09	1.31E- 09	0	501	0.035	0.000	0

49	Exp4_CR_sweep	4	rand	1.2	0.7	10	FALSE	-8.50E-11	1.12E-09	0	501	0.035	0.000	0
50	Exp4_CR_sweep	5	rand	1.2	0.7	10	FALSE	3.42E-10	4.17E-11	0	501	0.035	0.000	0
51	Exp5_dyn_amic_F	1	rand	U(0.5, 1.0)	0	10	TRUE	-0.9949586	7.20E-10	0.99495906	501	0.038	0.995	0
52	Exp5_dyn_amic_F	2	rand	U(0.5, 1.0)	0	10	TRUE	-1.33E-09	7.19E-10	0	501	0.037	0.000	0
53	Exp5_dyn_amic_F	3	rand	U(0.5, 1.0)	0	10	TRUE	-1.41E-11	-1.05E-09	0	501	0.037	0.000	0
54	Exp5_dyn_amic_F	4	rand	U(0.5, 1.0)	0	10	TRUE	1.39E-09	-1.51E-09	0	501	0.036	0.000	0
55	Exp5_dyn_amic_F	5	rand	U(0.5, 1.0)	0	10	TRUE	-1.38E-09	-1.13E-09	0	501	0.036	0.000	0